48

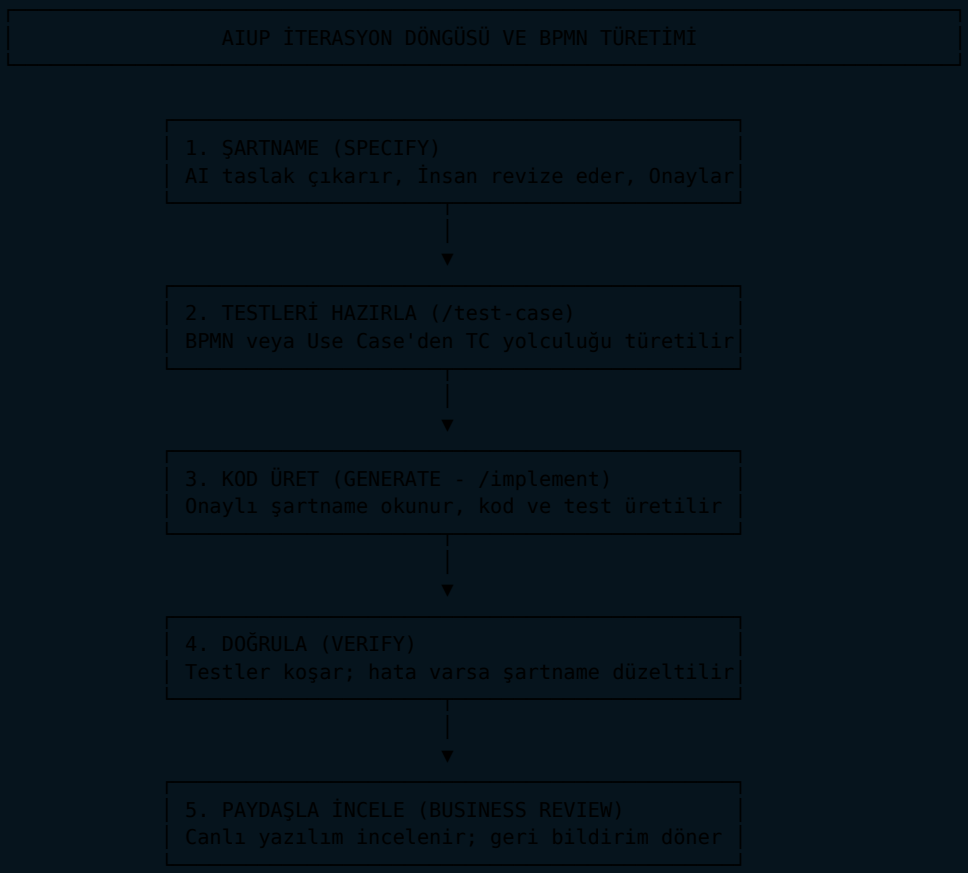
BÖLÜM: DILIMLER (SLICES), TESTLER VE İTERASYON DÖNGÜSÜ | MODULE: SLICES, TESTS, AND THE ITERATION LOOP

İterasyon Döngüsü ve BPMN Süreçlerinden Test Yolculuğu Türetme

The Iteration Loop & Deriving Journey Tests from BPMN Processes

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir film stüdyosunu düşünün: Önce senaryo yazılır (Şartname), sonra prova sahneleri planlanır (Testler), sonra aktörler sahneyi oynar (Kod üretimi), yönetmen sahneyi izler (Doğrulama), en son yapımcıya izletilir (Paydaş İncelemesi). Eğer yapımcı bir sahneyi beğenmezse aktör sette kafasına göre rol değiştirmez; senaryo yazarı senaryoyu günceller ve döngü baştan başlar!

🇬🇧 ENGLISH (ELI5)

Think of a film production studio: First the screenplay is polished (Spec), next the stunt sequences are choreographed (Tests), then actors perform on camera (Code Generation), the director reviews the footage (Verification), and finally the producer screens the cut (Business Review). If the producer wants a change, actors don't ad-lib on set; the writer updates the script and the loop re-executes cleanly!

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

AIUP İterasyon Döngüsü ve BPMN Entegrasyonu: Her AI Unified Process iterasyonu 5 adımlı katı bir döngüde koşar: 1) Belirt (Specify): Analist şartnameyi hazırlar, paydaş onaylar (Approved). 2) Testleri Hazırla: `/test-case` komutu çalıştırılır. Eğer Inception aşamasında bir BPMN iş süreci çizilmişse, bu süreçteki her bir aktivite otomatik olarak bir use case çağrısına, her bir karar dalı ise ayrı bir TC yolculuk testine dönüştürülür! 3) Üret (Generate): `/implement UC-XXX` komutu mevcut kodları yerinde uzlaştırarak (reconcile) yeni kodları üretir. 4) Doğrula (Verify): Otomatik testler çalışır. Bir test başarısız olursa sorun %90 şartnamedeki bir belirsizlikten kaynaklanmaktadır; şartname düzeltilir ve tekrar üretilir. 5) Paydaşla İncele: Çalışan yazılım paydaşa sunulur; geri bildirimler doğrudan şartnameye işlenir. Bu döngüde 'şartnameyi atlayıp koda yama yapmak' (hotfix bypass) kesinlikle yasaktır.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The 5-Step AIUP Iteration Loop & BPMN Derivation: Every engineering increment in the AI Unified Process executes a disciplined cadence: 1) Specify: The requirements engineer revises the use case to 'Approved'. 2) Prepare Tests: The `/test-case` engine ingests BPMN 2.0 process models from the Inception phase; every task activity maps to a use case invocation, and each gateway decision path synthesizes an independent TC journey file. 3) Generate: `/implement UC-XXX` executes, reconciling existing repositories and controllers in place without generating conflicting duplicates. 4) Verify: All tests execute automatically. Failures are traced back to spec ambiguities and rectified at the specification source. 5) Business Review: Working software is validated against the behavioral spec story. No code change ever bypasses this cycle.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: 5 Adımlı İterasyon Döngüsünün Programatik Temsili
package com.maang.aiup.iteration;

public class IterationLoopDemo {
    public enum SpecStatus { DRAFT, APPROVED, IMPLEMENTED, VERIFIED, DONE }

    public record SpecLifecycle(String useCaseId, SpecStatus status, boolean testsPassing) {
        public SpecLifecycle advance() {
            return switch (status) {
                case DRAFT -> new SpecLifecycle(useCaseId, SpecStatus.APPROVED, false);
                case APPROVED -> new SpecLifecycle(useCaseId, SpecStatus.IMPLEMENTED, false);
                case IMPLEMENTED -> new SpecLifecycle(useCaseId, SpecStatus.VERIFIED, true);
                case VERIFIED -> testsPassing
                    ? new SpecLifecycle(useCaseId, SpecStatus.DONE, true)
                    : new SpecLifecycle(useCaseId, SpecStatus.APPROVED, false);
                case DONE -> this;
            };
        }
    }

    void main() {
        var cycle = new SpecLifecycle("UC-002", SpecStatus.DRAFT, false);
        while (cycle.status() != SpecStatus.DONE) {
            System.out.println("🔄 Döngü Adımı: " + cycle.status());
            cycle = cycle.advance();
        }
        System.out.println("🎯 İterasyon Tamamlandı: Use Case DONE durumuna ulaştı!");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Üretim baskısı altında bile 'kodu hemen düzeltelim, şartnameyi sonra güncelleriz' tuzağına düşmeyin! Şartnameyi atlayan her müdahale şartnamenin güvenilirliğini öldürür.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never fall into the trap of patching code directly under production pressure with promises to update the spec later. Bypassing the specification starts the rot.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- AIUP iterasyon döngüsü 5 adımdan oluşur: Specify -> Test -> Generate -> Verify -> Review.
- Inception aşamasındaki BPMN süreçleri otomatik olarak TC-XXX yolculuk testlerine dönüştürülür.
- Test başarısızlığı öncelikle şartnamedeki bir eksiklik olarak ele alınır ve şartnamede düzeltilir.

🎯 Key Architectural Takeaways

- The AIUP iteration loop runs 5 steps: Specify -> Test -> Generate -> Verify -> Review.
- BPMN business process models directly synthesize executable TC-XXX test journeys.
- Test failures indicate spec ambiguities first and are rectified at the specification source.